DOCQUERY AI

Repository Theory Guide

A practical explanation of the RAG pipeline, code architecture, retrieval algorithms, grounded generation, evaluation, and engineering trade-offs.

Repository reviewed: github.com/ApexHyper2007/DocQuery-AI

Source files reviewed include README.md, PROJECT_REPORT.md, retrieval.py, answering.py, ollama_client.py, api.py, and evaluate_retrieval.py. The guide emphasizes what the repository actually implements and separates that from suggested future work.

Study goal: be able to trace a question from the UI to retrieval, explain why a chunk was selected, and describe what evidence supports the final answer.

Prepared 20 October 2026 | Version-aware notes for the public main branch

Contents and study map

Read chapters 1-10 for the data pipeline; 11-16 for search and answering; 17-21 for serving and testing; 22-29 for evaluation, reliability, limitations, and viva preparation.

1. What DocQuery AI is
2. RAG: why retrieval is part of the answer
3. The complete system architecture
4. Reading the repository structure
5. Data ingestion and source provenance
6. Redirects, malformed pages, and ingestion failures
7. HTML cleaning and Markdown conversion
8. Structure-aware and code-aware chunking
9. Chunk IDs, metadata, and traceability
10. Embeddings and semantic similarity
11. ChromaDB and persistent vector storage
12. BM25: exact keyword retrieval
13. Dense retrieval and candidate ranking
14. Hybrid search and reciprocal-rank fusion
15. Grounded generation with Ollama
16. Evidence thresholds, citations, and abstention
17. FastAPI: the backend contract
18. Streamlit: the user-facing interface
19. Configuration and runtime dependencies
20. Windows setup and end-to-end execution
21. Testing: unit tests and integration boundaries
22. Retrieval evaluation: hit rate and MRR
23. Understanding key retrieval metrics
24. Failure analysis: diagnose the right stage
25. Privacy, security, and operational boundaries
26. Performance and resource trade-offs
27. Limitations and a credible roadmap
28. Viva questions and model answers
29. Command reference and source links

> **Accuracy note: the README reports six indexed documentation pages and 1,000 chunks in the local index. The published evaluation examples are described as a smoke benchmark, not a representative quality guarantee.**

1. What DocQuery AI is

DocQuery AI is a local Retrieval-Augmented Generation (RAG) assistant for technical questions about official PyTorch, torchvision, and NVIDIA CUDA documentation. Instead of asking a language model to answer from memory alone, the system searches a prepared documentation corpus and sends the most relevant passages to a local Ollama model.

The problem it addresses

Technical documentation is accurate but often fragmented across pages, headings, tables, and code examples. A developer may remember the idea behind an API but not the exact name, syntax, caveat, or version-specific detail. Keyword search can miss paraphrased questions; an LLM without retrieved evidence may confidently state something that is outdated or unsupported.

What the application does

- Builds a controlled corpus from six official documentation pages.
- Cleans HTML into a Markdown-like representation, retaining headings, links, tables, lists, and code blocks.
- Splits content into small, metadata-rich chunks and embeds them with sentence-transformers/all-MiniLM-L6-v2.
- Stores vectors persistently in ChromaDB and combines dense retrieval with a local BM25 keyword ranker.
- Uses reciprocal-rank fusion (RRF) to combine rankings, then asks Ollama to produce a source-grounded answer.
- Provides a FastAPI backend and Streamlit chat interface, including visible source excerpts.

> **The core project is a retrieval-and-answer pipeline, not a model trained from scratch.**

The README reports an index of 1,000 chunks. It also explicitly says the six-question benchmark is only a smoke benchmark: it demonstrates a test path, not a statistically strong claim about answer quality.

2. RAG: why retrieval is part of the answer

Retrieval-Augmented Generation separates finding information from writing a response. The retrieval stage finds passages from an external knowledge source; the generation stage reads those passages with the user question and produces an answer.

Generation without retrieval

A general language model predicts text from patterns learned during training and from the current prompt. It might answer a PyTorch question plausibly, but the answer may not reflect the documentation version indexed by this project. It may also fill gaps with details that were never in the source.

Retrieval before generation

```
Question -> retrieve passages -> construct context
        -> send question + context to LLM
        -> answer with source labels
```

At runtime, retrieved excerpts serve as temporary evidence. They are not permanently learned by the model. Updating the corpus and rebuilding the index can therefore change the evidence available to answer without fine-tuning the LLM.

What RAG does not guarantee

- The retriever may miss the relevant section.
- A chunk may omit the caveat or code example needed to interpret a statement.
- The LLM may misread context or produce a claim not supported by it.
- A citation label can be formatted correctly while still failing to support the sentence it follows.

> **RAG improves access to relevant evidence; it does not automatically guarantee truthfulness or correctness.**

DocQuery addresses these risks with hybrid search, source metadata, a grounding-oriented prompt, and an insufficient-context path. Each still needs evaluation.

3. The complete system architecture

```
OFFLINE / INDEX BUILD
Official docs -> ingest_docs.py -> raw HTML + manifest
             -> process_docs.py -> Markdown-aware chunks
             -> index_docs.py -> MiniLM vectors -> ChromaDB

ONLINE / QUESTION ANSWERING
Question -> BM25 + dense Chroma search -> RRF fusion
         -> evidence thresholds -> Ollama grounded response
         -> FastAPI response -> Streamlit UI + source excerpts
```

Offline versus online work

Ingestion, cleaning, chunking, and embedding build the knowledge base. They are relatively expensive and do not need to run for every question. At query time, the system loads the processed chunks for BM25, connects to the persistent Chroma collection, embeds the new question, ranks candidates, checks evidence, and may call the LLM.

Why the split matters

A query should not re-download and re-embed every source page. Separating stages improves latency and makes failures easier to diagnose. A bad answer can be investigated at each boundary: source ingestion, clean text, chunk boundaries, embedding/index consistency, retrieval ranking, threshold decision, prompt, or model response.

Component contracts

- Ingestion produces validated raw pages and a manifest.
- Processing produces chunks with stable IDs and citation metadata.
- Indexing stores vectors and documents in the expected collection.
- Retrieval returns ranked chunks with method-specific scores.
- Answering returns an answer, insufficient_context flag, and source objects.
- API exposes a validated request/response interface; UI renders that response.

These contracts are useful for unit testing: an individual stage can be tested without running the whole application.

4. Reading the repository structure

The repository is organized around clear pipeline stages. Start with README.md for the intended behaviour, then PROJECT_REPORT.md for the stage-by-stage engineering rationale, and finally read the corresponding source files in execution order.

Path	Purpose
src/ingest_docs.py	Allowlisted downloads, redirect resolution, validation, manifest
src/process_docs.py	HTML cleaning, Markdown rendering, section/code-aware chunking
src/index_docs.py	MiniLM embedding generation and persistent Chroma storage
src/retrieval.py	BM25, dense search, and RRF rank fusion
src/answering.py	Evidence thresholds, abstention, source records, answer orchestration
src/ollama_client.py	Local Ollama /api/chat request and grounded prompt
src/api.py	FastAPI health/query endpoints and validation
src/streamlit_app.py	Chat UI and expandable supporting excerpts
src/evaluate_retrieval.py	Source-level hit-rate and MRR evaluation
tests/	Unit tests and six manually labeled starter questions
README.md / PROJECT_REPORT.md	Setup guide, configuration, rationale, caveats

How to read a source file

Identify its input and output first. Then find configuration defaults, validation checks, the core algorithm, error paths, and tests. For example, retrieval.py accepts a question plus k/candidate_k/rrf_k and returns ranked chunk dictionaries; answering.py consumes them and returns the API-facing answer structure.

> **Debug one stage at a time. Do not change the LLM prompt to compensate for a corpus or retrieval defect.**

5. Data ingestion and source provenance

The ingestion stage downloads a fixed allowlist of technical documentation pages, rather than crawling arbitrary websites. For a documentation assistant, source selection is part of the quality and safety boundary: whatever gets ingested may later influence the answer.

Why an explicit allowlist?

- Keeps the corpus focused on official PyTorch, torchvision, and NVIDIA CUDA material.
- Makes refreshes more reproducible because the expected inputs are known.
- Reduces the chance of fetching unrelated pages, user-generated content, or unsafe destinations.
- Makes source attribution easier because every chunk can point back to a known URL.

Validation and manifest

src/ingest_docs.py records ingestion outcomes in data/raw/manifest.json. The project report describes validation for redirect stubs, unsafe hosts, loops, oversized or non-HTML responses, and preservation of requested versus final URLs. This is important because the HTTP response can succeed while the saved page still contains no real documentation.

Data provenance

Provenance means retaining where the content came from, how it was acquired, and enough identifying information to trace a result back to its source. In DocQuery, provenance supports debugging, citations, change detection, and review of licensing or attribution requirements.

> **Data quality propagates downstream: a bad page becomes bad Markdown, then a bad chunk, then a bad retrieval candidate, and potentially a bad answer.**

A robust refresh workflow should compare final URLs, content hashes, page sizes, and the count of successfully processed chunks before replacing a previously valid index.

6. Redirects, malformed pages, and ingestion failures

Not every successful HTTP response is the content the application wanted. Some documentation sites serve a short HTML page containing JavaScript or a meta-refresh directive that redirects the browser to a versioned URL. A downloader that saves only the initial response may accidentally index this redirect shell.

Redirect types

- HTTP redirect: the server responds with a 3xx status and a Location header; the HTTP client can follow it.
- JavaScript redirect: client-side script changes the browser location.
- Meta refresh: HTML instructs the browser to navigate after a specified delay.
- Canonical or anchor fallback: a redirect page may reveal its intended destination through a link or page metadata.

Why safe redirect handling matters

The project report says the ingestion logic resolves supported redirect forms relative to the response URL and validates destinations against allowed hosts. It also guards against loops and non-document responses. Relative resolution is essential because a target like ../../2.14/page.html depends on the redirect page path.

Useful operational checks

- Open the first few lines of each raw HTML page and make sure it is actual documentation.
- Confirm configured URL and final URL in the manifest.
- Review failed items before running processing or indexing.
- If a downloaded page is tiny or contains a redirect title, fix ingestion instead of trying to tune embeddings.

```
PowerShell quick inspection:
Get-Content .\data\raw\manifest.json
Get-Content .\data\raw\pytorch\some-page.html -TotalCount 20
```

The principle generalizes beyond RAG: make upstream data assumptions explicit and test them.

7. HTML cleaning and Markdown conversion

Official documentation pages contain main article text alongside navigation, sidebars, scripts, styling, breadcrumbs, and repeated page controls. If all HTML text is embedded, irrelevant navigation can dominate chunks and waste retrieval capacity.

Cleaning goals

- Remove page furniture such as scripts, styles, navigation, and irrelevant UI containers.
- Retain headings because they describe what a paragraph or example is about.
- Retain code blocks, parameter descriptions, lists, links, and tables when possible.
- Normalize the page to a consistent representation before splitting it.

Why Markdown is an intermediate format

Markdown is easy for both humans and downstream code to inspect. It preserves high-value structure with relatively little markup: heading levels, fenced code, links, lists, and tables. A chunk can then carry a heading path such as a page title followed by nested section names, giving the LLM context beyond isolated sentences.

Repository implementation

src/process_docs.py uses BeautifulSoup for parsing and renders selected document elements into Markdown. The project report notes article/main-root selection, removal of unwanted tags, conversion of common content types, resolution of relative links against the final page URL, and retention of the heading path.

What to inspect when quality is poor

Compare raw HTML with the processed Markdown. Look for missing parameter tables, code blocks broken into prose, repeated navigation text, incorrectly joined paragraphs, or links that point to the wrong version. Do this before adjusting the vector-search settings.

> **The embedding model can only represent the text it receives. It cannot recover content that processing accidentally deleted.**

8. Structure-aware and code-aware chunking

Chunking divides a page into smaller retrieval units. The chunk must be large enough to retain a complete idea but small enough to focus search and fit within embedding and LLM context limits.

DocQuery defaults

- Target chunk size: 450 characters.
- Target overlap: 60 characters.
- General separators prioritize blank lines, then newlines, then spaces.

These are implementation defaults rather than universal optimal settings. The report describes heading-aware section splitting outside fenced code and a code-aware packing step that avoids needlessly breaking code fences and repeats the section heading around chunks.

Trade-offs

Too small	Too large
Precise topic focus; more chunks	More local context per result; fewer chunks
May split API description from parameter caveat	Can mix multiple topics and dilute relevance
More metadata and index overhead	Higher risk of embedding truncation or long prompts

Why headings matter in code documentation

A code snippet like `x.add(y)` is ambiguous without a section describing tensor semantics, broadcasting, types, or return values. Repeating the nearest heading adds a compact topic label to the snippet. Similarly, keeping fenced examples intact helps a reader understand which lines belong together.

Overlap is a compromise

Overlap copies a short span into the next chunk so information near a boundary is less likely to disappear from every chunk. Excessive overlap produces near-duplicate hits, consumes storage, and can crowd out different evidence during top-k retrieval. Evaluate chunk size and overlap on a fixed question set instead of relying on intuition alone.

9. Chunk IDs, metadata, and traceability

A retrieved paragraph is more useful when the application also knows where it came from. DocQuery keeps chunk content with a compact set of identifiers and document metadata so that retrieval, answer generation, source display, and evaluation can refer to the same passage.

Metadata described by the project

- chunk_id and chunk position/count.
- Source identifier, page title, section title, and heading path.
- Source URL and configured URL, preserving provenance across redirects.
- Source file and character count, plus hash/timestamp information useful for debugging.

Why stable IDs are important

Hybrid retrieval can return the same chunk from both BM25 and dense search. A stable chunk_id provides a join key so RRF can combine these appearances into one candidate. IDs also help make index updates idempotent: re-indexing a chunk can update the existing record instead of creating an untraceable duplicate.

Traceability through the pipeline

```
source URL
   -> processed chunk + heading metadata
   -> vector document / BM25 candidate
   -> fused result with scores and retrieval methods
   -> source object S1 / S2 shown with answer
```

Metadata must be preserved carefully when two search systems return the same chunk. The code in retrieval.py keeps dense distance/similarity and BM25 score fields when fusing candidates, while public source objects in answering.py expose a selected metadata set rather than leaking every internal field.

> **If an answer looks wrong, the source URL, heading path, chunk text, and retrieval scores are the first things to inspect.**

10. Embeddings and semantic similarity

An embedding model maps text to a numerical vector. Similarity search compares vectors so a question can retrieve conceptually related text even when the wording differs. In DocQuery, sentence-transformers/all-MiniLM-L6-v2 is used for both document chunks and questions.

The intuition

A question like “How do I add two tensors?” may be related to documentation discussing `torch.Tensor.add`, even if the question uses an everyday paraphrase. A lexical search may miss that wording; an embedding can capture part of the semantic relationship learned during model training.

Important model facts in this repository

- Model: sentence-transformers/all-MiniLM-L6-v2.
- Vector dimensionality recorded by the project: 384.
- Embeddings are normalized and converted to NumPy arrays before persistence.
- The project report calls out a 256-token input limit, so chunks are kept small and checked for token-limit risk.

Embeddings are not truth scores

A vector score estimates similarity between representations, not whether a statement is true, complete, or relevant enough to answer the exact question. Two passages can be semantically close but differ in API version, parameter, platform, or important caveats.

Consistency requirements

- Use the same compatible model to embed documents and questions.
- Use the expected vector dimension and distance configuration.
- Rebuild the index if the embedding model changes; old and new vector spaces may not be comparable.

In this project, CPU is the default device; `DOCQUERY_DEVICE=cuda` is an explicit opt-in for embedding. GPU use should be validated for speed and memory rather than assumed to be automatically better.

11. ChromaDB and persistent vector storage

ChromaDB stores the document vectors and supports nearest-neighbour search. In DocQuery, the persistent collection is named `docquery_docs`; records include chunk IDs, document text, metadata, and embeddings.

Indexing workflow

- Load processed records from data/processed/chunks.jsonl.
- Validate chunk text and token limits against the embedding model.
- Encode chunks in batches with normalized embeddings.
- Upsert identifiers, content, metadata, and vectors into Chroma.
- Write index metadata such as model name, embedding dimension, corpus hash, and counts.

Why persist the index?

Without persistence, the application would need to load all documents and calculate every embedding again on each start. Persistent storage separates the build step from query-time retrieval and makes repeated local runs practical.

Distance versus similarity

The project queries Chroma for distances and computes `dense_similarity = 1 - distance`. That value is used by the app as a ranking/evidence heuristic. It must not be presented as a calibrated probability of correctness, and score interpretation depends on the metric and normalization configuration.

Rebuild safety

The `--rebuild` option is documented as resetting only the named DocQuery collection. Restricting the destructive action to the collection owned by this project is safer than resetting an entire vector database.

> **Index health is a consistency problem: processed chunk IDs, Chroma IDs, embedding model, vector dimensions, and index metadata should agree.**

12. BM25: exact keyword retrieval

BM25 is a lexical retrieval algorithm. It ranks a chunk partly by the frequency of query terms in that chunk, the rarity of each term across the corpus, and normalization for document length. It does not create vectors and does not need a language model.

Why DocQuery includes BM25

Technical questions often contain exact identifiers such as `torch.Tensor.add`, parameter names, flags, and acronyms. Dense embeddings can blur punctuation or retrieve conceptually similar sections. BM25 is often strong when an exact symbol or rare term is important.

Core formula (conceptual form)

```
BM25(D,Q) = sum over query terms t of
  IDF(t) * [ f(t,D) * (k1 + 1) ]
           / [ f(t,D) + k1*(1-b+b*|D|/avgdl) ]
```

Here, f(t,D) is term frequency in the chunk, |D| is its token length, avgdl is average chunk length, k1 controls term-frequency saturation, and b controls how strongly document length is normalized. The implementation defaults are k1=1.5 and b=0.75.

Code implementation

In src/retrieval.py, TOKEN_RE is designed to preserve punctuation-bearing technical identifiers. The tokenizer lowercases tokens; BM25Index precomputes token lists, lengths, per-chunk frequency counters, average length, document frequency, and IDF. `search()` sorts by score and removes zero-score results.

Interpretation caution

BM25 scores are corpus- and query-dependent. A score of 2 is not “2 out of 10 confidence,” and it cannot be directly added to cosine distance without a deliberate calibration strategy.

13. Dense retrieval and candidate ranking

Dense retrieval embeds the question and searches the Chroma collection for nearby document vectors. It is useful when a user describes an idea with different vocabulary than the original source.

Query-time flow in retrieval.py

- Reject an empty question and invalid k/candidate_k values.
- Get BM25 candidates from the in-memory chunk index.
- Confirm the Chroma collection is non-empty.
- Encode the question with the same MiniLM model used for the corpus.
- Ask Chroma for up to candidate_k documents, metadata, and distances.
- Attach `dense_distance` and the derived `dense_similarity` to each returned candidate.
- Fuse the lexical and dense lists and return the first k results.

candidate_k versus k

candidate_k controls how many possible candidates each retrieval method can contribute to fusion. `k` controls how many fused results leave the retriever for downstream answering. A larger candidate pool can improve recall; a smaller final set reduces prompt length and noise.

Common failure modes

- The answer passage exists in raw docs but was dropped during cleaning.
- The passage is split in a way that loses its topic or caveat.
- Embedding similarity favours a related but non-answering section.
- The Chroma index was built with a different corpus or model than the live retriever expects.

Dense search is one signal, not a substitute for exact-match search, provenance inspection, or labeled relevance evaluation.

14. Hybrid search and reciprocal-rank fusion

Hybrid retrieval combines lexical and semantic rankings. DocQuery obtains BM25 results from a small in-process implementation and dense results from Chroma, then merges their rankings using Reciprocal Rank Fusion (RRF).

Why not just add scores?

BM25 scores and vector distances have different meanings and scales. Adding them directly would require a normalization/calibration design. RRF avoids that issue by using each candidate's rank position rather than its raw score.

RRF formula

```
RRF_score(d) = sum over result lists r of
                 1 / (rrf_k + rank_r(d))
```

`rank_r(d)` is the one-based position of chunk d in ranking r. The project default is `DOCQUERY_RRF_K=60`. Candidates found near the top of multiple lists tend to accumulate larger fusion scores. The fusion score is a ranking value, not a probability.

Implementation details

- Candidates are keyed by stable chunk_id.
- Duplicate IDs inside a single ranking are ignored.
- The entry records whether BM25 and/or dense search found it.
- Dense similarity/distance and BM25 score metadata are retained where present.
- Sorting is deterministic, using fused score and then chunk_id for ties.

How to tune it

Vary candidate_k, final top_k, and rrf_k on a fixed set of questions. Inspect failure cases: is the useful passage absent from both candidate lists, ranked too low in one list, or crowded out by duplicate chunks? That diagnosis is more useful than blindly increasing every parameter.

15. Grounded generation with Ollama

Ollama is the local model runtime. DocQuery does not bundle model weights inside the Python application; it sends an HTTP request to a running Ollama service. The default model in the README is `llama3.2:3b`.

What the client sends

src/ollama_client.py builds a prompt containing source labels such as [S1], a source title/URL, and the retrieved excerpt text. It instructs the model to answer using only supplied documentation, cite factual claims, avoid inventing API behavior or parameters, and admit when excerpts are insufficient.

Request mechanics

```
POST http://localhost:11434/api/chat
  model: llama3.2:3b
  stream: false
  messages: system instruction + user question/context
  options: temperature = 0.1
```

The client uses Python's standard library (`urllib.request`) and JSON serialization. It handles HTTP errors, connection failures, invalid JSON, and empty responses by raising an OllamaError that the API can translate into a service error.

Why low temperature?

A low temperature reduces response variability for factual tasks. It does not make an answer automatically factual; correctness still depends on retrieved context, model capability, and how well the answer follows the evidence constraints.

Runtime and hardware

The local setup requires Ollama to be installed separately and the model to be pulled. Model size, quantization, RAM/VRAM, and available compute affect latency. The documented model can be changed through `DOCQUERY_OLLAMA_MODEL` to fit the machine budget.

> **Local inference avoids a hosted LLM API in this documented path, but does not remove the need to secure the machine and inspect runtime configuration.**

16. Evidence thresholds, citations, and abstention

An assistant should not be forced to answer every question. DocQuery's answering layer applies an insufficient-context path when it has no results or when the best dense and lexical signals are both below configured thresholds.

The logic in answering.py

```
results = retriever.search(question, ... )
if no results: return insufficient context

best_similarity = max(dense_similarity)
best_bm25 = max(bm25_score)
if both are below thresholds:
    return insufficient context
else:
    assign S1, S2, ... source labels
    call grounded generator
```

Default thresholds

- DOCQUERY_MIN_SIMILARITY = 0.20
- DOCQUERY_MIN_BM25_SCORE = 1.5

These are heuristics, not calibrated probabilities. The logic uses an OR-style acceptance: if either signal reaches its threshold, the answer path may continue. This can preserve exact keyword matches that have lower semantic similarity and paraphrases that have weaker exact-term scores.

Two important error types

- False accept: weak context passes and an unsupported answer is generated.
- False abstention: useful context is found but the thresholds reject it.

Citation behaviour

The application assigns source IDs to retrieved results and returns metadata including chunk ID, section title, heading path, URL, content, and scores. The prompt asks the model to cite factual claims using the labels. Citation presence is not the same as citation entailment: evaluation must check whether the source truly supports each claim.

> **Tune thresholds with labeled relevant and irrelevant queries, and measure both false accepts and false abstentions.**

17. FastAPI: the backend contract

FastAPI provides the HTTP boundary between the question-answering pipeline and the front end. It also makes the project easier to test using a REST client, PowerShell, or the automatically generated OpenAPI documentation.

Endpoints

- GET /health returns a small health response.
- POST /api/query accepts question, top_k, and candidate_k and returns answer, insufficient_context, and sources.

QueryRequest in src/api.py uses Pydantic Field constraints: question length is 1 to 4,000 characters, top_k is 1 to 20, and candidate_k is 1 to 100. This is input validation at the service boundary; invalid values should not proceed into indexing or retrieval code.

Initialization and error translation

The retriever is loaded lazily and cached with lru_cache so the model and retrieval resources are not reconstructed for every request. Errors are mapped to useful HTTP statuses: Ollama errors become 502, missing/unready retrieval data and value errors become 503, and unexpected errors become 500.

Health is not readiness

The /health endpoint reports that the API process responds. It does not necessarily prove that the Chroma collection exists, the embedding model is cached, or Ollama is reachable. A production-ready deployment would often separate liveness from dependency readiness.

```
PowerShell smoke test:
Invoke-RestMethod http://127.0.0.1:8000/health

Invoke-RestMethod -Method Post `
  -Uri http://127.0.0.1:8000/api/query `
  -ContentType "application/json" `
  -Body {"question":"What does torch.Tensor.add do?"}
```

18. Streamlit: the user-facing interface

Streamlit supplies the local chat interface without requiring a separate front-end build system. It collects a user question, calls the FastAPI backend, displays the answer, and exposes the retrieved source excerpts to users.

Why separate UI and API?

- The retrieval pipeline can be tested independently of the interface.
- The UI can be replaced later without rewriting the core retrieval and generation logic.
- Backend and frontend failures can be isolated during debugging.
- The API becomes a stable contract for future clients.

Source display is part of trust

Expandable source excerpts show which sections and URLs contributed to a response. That gives users a way to verify a claim, inspect a missing caveat, or identify a retrieval mismatch. It also helps developers report bugs with concrete evidence instead of saying only that “the AI got it wrong.”

Current scope limitation

The project report describes the current UI as local and single-user, without authentication or concurrent-session persistence. That is an understandable first-project scope but it is not a full multi-user production application.

Potential improvements

- Show each source’s rank signals and make the selected snippet easy to inspect.
- Make errors actionable (for example, explain if Ollama is offline).
- Add loading and timeout feedback for slower local models.
- Add conversation persistence only after deciding how histories should be stored and protected.

19. Configuration and runtime dependencies

The project uses environment variables for values that may differ by machine or experiment. Defaults make local setup easier, while overrides let users choose a model, collection, device, or retrieval threshold without editing source code.

Variable	Documented role / default
DOCQUERY_OLLAMA_URL	Local Ollama base URL; http://localhost:11434
DOCQUERY_OLLAMA_MODEL	Generation model; llama3.2:3b
DOCQUERY_EMBEDDING_MODEL	sentence-transformers/all-MiniLM-L6-v2
DOCQUERY_DEVICE	Embedding device; cpu by default
DOCQUERY_COLLECTION	Chroma collection; docquery_docs
DOCQUERY_RRF_K	RRF constant; 60
DOCQUERY_MIN_SIMILARITY	Dense evidence threshold; 0.20
DOCQUERY_MIN_BM25_SCORE	Lexical evidence threshold; 1.5
DOCQUERY_BM25_K1 / _B	BM25 tuning; 1.5 / 0.75

Python packages versus external runtime

The Python environment installs the application dependencies listed in requirements.txt. Ollama must be installed separately, then the chosen model must be pulled. A successful `pip install` does not imply that the local language model server is running.

Secrets and local settings

The README says .env is ignored by Git and .env.example contains non-secret defaults. Avoid committing credentials or private configuration. Do not assume that environment variables set in one PowerShell process are automatically present in a different terminal window.

> **When a setting changes the embedding model, collection, or index format, check whether rebuilding the index is required.**

20. Windows setup and end-to-end execution

The README documents a Windows PowerShell workflow. The safest starting point is an isolated virtual environment to avoid altering another Python project's dependencies.

1. Create environment and install packages

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

2. Build or refresh the knowledge base

```
python .\src\ingest_docs.py --refresh
python .\src\process_docs.py
python .\src\index_docs.py
```

Ingestion downloads the current source allowlist; processing writes chunks; indexing embeds and persists them. If a valid index already exists, the README says these stages can be skipped for normal query use.

3. Start Ollama

```
ollama pull llama3.2:3b
ollama serve
```

4. Start API and UI in separate terminals

```
uvicorn src.api:app --host 127.0.0.1 --port 8000

streamlit run .\src\streamlit_app.py
```

5. Verify the service

Open the Streamlit URL printed in the terminal. The API docs are at http://127.0.0.1:8000/docs while the API is running. Test /health first, then POST a real question. If the API works but answers fail, check retrieval files and Ollama separately.

> **Useful debugging order: environment and imports -> index files -> Chroma collection -> Ollama server/model -> API -> Streamlit.**

21. Testing: unit tests and integration boundaries

Tests check expected behavior at a particular boundary. They do not prove that the overall application is accurate, but they make regressions easier to identify when the code changes.

Tests listed by the project report

- tests/test_ingest_docs.py: redirect resolution and redirect-stub rejection.
- tests/test_retrieval.py: API tokenization, BM25 ranking, RRF agreement, citation metadata, and weak-context refusal.
- tests/test_api.py: health response and query response shape without requiring Ollama.

Why not call the LLM in every unit test?

LLM calls can be slower, depend on local runtime state, and produce variable text. A unit test can inject a fake retriever or generator and check that the application calls the right boundary with the right inputs. This keeps many tests fast and deterministic. A smaller integration suite can exercise the actual local runtime when needed.

Run the tests

```
python -m pytest -q
```

Examples of useful assertions

- A blank query is rejected.
- A punctuation-bearing API symbol remains searchable as expected.
- A chunk appearing in both rankings is returned once after fusion.
- Source metadata survives the path to the public response.
- Weak evidence does not call the generator when the refusal condition is met.

Treat a passing test suite as evidence that specified invariants hold, not as a substitute for evaluating relevance, factual support, and answer completeness on real questions.

22. Retrieval evaluation: hit rate and MRR

Retrieval evaluation asks whether the system places relevant material near the top of the ranked list. The repository provides six manually labeled source-level questions in tests/eval_questions.json and a command that reports hit rate at k and Mean Reciprocal Rank (MRR).

Hit rate at k

Hit@k is 1 when at least one expected source appears in the top k results and 0 otherwise. Averaging across questions gives the share of questions for which retrieval found a labeled source within the top k.

Mean Reciprocal Rank

```
RR for a query = 1 / rank of first relevant result
MRR = average of RR across all queries
```

If the first relevant source is ranked first, reciprocal rank is 1. If it appears at rank 5, reciprocal rank is 0.2. If no relevant source appears in the evaluated list, the score is 0.

Run the evaluator

```
python -m src.evaluate_retrieval --k 5
```

Important limitation of the present benchmark

The evaluator checks source-level labels (source plus title slug), not whether the exact retrieved chunk fully answers the question. Six queries are too few to support broad quality claims. The README explicitly calls this a smoke benchmark.

A stronger next benchmark

- Expand the query set across exact API lookups, conceptual questions, code examples, and answer-not-in-corpus cases.
- Label relevant chunk IDs, not just source pages.
- Report Recall@k, MRR, and NDCG; inspect individual misses.
- Separately judge answer faithfulness, correctness, citation support, and completeness.

23. Understanding key retrieval metrics

A single metric cannot fully characterize a RAG system. Different metrics answer different questions, so use a small set and pair it with failure review.

Recall@k

Recall@k measures how much of the known relevant material was retrieved among the top k. If three chunks are labeled relevant and two appear in the retrieved top-k set, recall is 2/3. It is useful when missing evidence is costly.

Precision@k

Precision@k measures the fraction of the top k results that are relevant. High recall with low precision means the system finds useful evidence but also brings a lot of noise; that noise can distract generation.

MRR

MRR rewards putting the first relevant result high in the list. It is convenient for question-answering cases where one good passage near the top can be particularly valuable.

NDCG

Normalized Discounted Cumulative Gain supports graded relevance labels and rewards highly relevant results appearing near the top. It is useful when passages can be “partly relevant” or “strongly relevant,” rather than strictly relevant/irrelevant.

Generation-side metrics are separate

- Faithfulness: are claims supported by the retrieved context?
- Answer correctness: does the answer accurately resolve the user question?
- Citation precision: do cited sources support their associated claims?
- Abstention quality: does the system refuse when evidence is insufficient, without refusing too often?

> **Measure retrieval quality and answer quality separately. A good retriever can still be paired with a poor generator, and vice versa.**

24. Failure analysis: diagnose the right stage

When an answer is wrong, first determine what kind of failure occurred. Changing the prompt for every failure often hides the root cause and makes the system harder to reason about.

Symptom	Likely layer	Diagnostic action
Chunk is empty or redirect text	Ingestion	Inspect raw HTML and manifest; fix redirect validation
Parameter table missing	Processing	Compare raw HTML and processed Markdown
Question topic split from code	Chunking	Inspect boundaries, section labels, overlap
Exact API symbol is missed	Lexical retrieval	Check tokenization and BM25 output
Paraphrase is missed	Dense retrieval	Inspect query embedding and nearest neighbours
Relevant hit too low after merge	RRF/candidates	Inspect both rankings and candidate_k
Wrong claim despite good context	Generation/grounding	Inspect prompt, context order, model output, citations
Should have refused but answered	Thresholding	Label negatives; tune thresholds and OR logic
API returns 502	Ollama client/runtime	Check URL, runtime process, model, timeout

A disciplined debugging loop

- Save the exact question and expected supporting source.
- Inspect processed chunks matching the question or source.
- Print BM25 and dense rankings before RRF.
- Inspect the fused top results and threshold signals.
- Inspect the full prompt/context and returned answer.
- Add a regression test or evaluation case after fixing the cause.

This turns each failure into reusable engineering knowledge rather than a one-off manual correction.

25. Privacy, security, and operational boundaries

The README states that the documented setup sends questions and retrieved excerpts to a local Ollama service on the same machine and does not send them to a hosted model in this code path. That is a useful privacy property, but it is not equivalent to a complete security audit.

Data and repository hygiene

- Raw downloads, processed corpora, the vector database, virtual environment, model cache, and secrets are ignored by Git.
- The `.env.example` file should contain non-secret defaults only.
- Review `git status --short` and `git diff --check` before publishing.
- Check the license and attribution terms of source material before redistributing scraped documentation.

Local does not mean risk-free

A local server can still expose data if bound to a network interface, if another process can access the machine, or if the Ollama URL is configured to a remote host. Verify the effective configuration before using private documentation.

Production gaps described by the report

- The interface is a local single-user experience without authentication.
- There is no concurrent-session persistence in the first version.
- Health status does not fully validate every dependency.
- More comprehensive retrieval and answer evaluation is needed.

Before deployment, define the threat model, add authentication and rate limiting where appropriate, restrict network exposure, handle uploaded/private data deliberately, and avoid logging sensitive excerpts unnecessarily.

26. Performance and resource trade-offs

A RAG request involves several costs: loading/using the embedding model, lexical scoring, vector search, constructing a prompt, and generating tokens through Ollama. The slowest part on a local laptop is often LLM generation, but it should be measured rather than assumed.

Current design choices

- Embeddings default to CPU; CUDA is an explicit configuration choice.
- Document vectors are generated in batches and persisted.
- BM25 runs locally in memory without a separate search service.
- ChromaDB avoids re-embedding the corpus for every query.
- A small local model is used to fit more modest hardware budgets.

Measure before optimizing

- Ingestion duration and number of valid pages.
- Chunk counts and distribution of chunk lengths.
- Index-build time and persistent storage size.
- BM25 latency, dense-search latency, and fusion latency.
- Time to first token / complete answer, memory usage, and failure rate.
- Retrieval and answer quality before and after a configuration change.

Candidate count is also a performance parameter

Increasing candidate_k can help retrieval recall but increases fusion work and may change the result set. Increasing top_k adds more context and may improve completeness, but also increases the prompt and can distract a small model. Keep benchmark queries fixed when comparing these changes.

> **Optimize quality, latency, and memory together; the fastest configuration is not necessarily the most useful.**

27. Limitations and a credible roadmap

The repository is a solid local technical-documentation RAG prototype, but its scope and evidence should be described accurately. Six source pages and a small smoke benchmark do not establish broad coverage or production-grade reliability.

Current boundaries

- Corpus coverage is limited to six official documentation pages and may miss relevant material elsewhere in the docs.
- The existing local index is reported as 1,000 chunks; it is a generated artifact and excluded from Git.
- The six-query evaluator uses source-level relevance labels rather than fully labeled relevant chunks.
- Thresholds are heuristics and have not been demonstrated as calibrated confidence.
- Local single-user UI, no authentication, and no session persistence are documented constraints.

Improvement sequence

- Expand coverage carefully and validate every acquired page; avoid indiscriminate crawling.
- Build a representative benchmark with relevant chunk IDs and negative/out-of-corpus questions.
- Track Recall@k/MRR/NDCG, citation support, answer faithfulness, and abstention quality.
- Add answer-level regression cases for exact API names, versions, parameters, and code examples.
- Add observability: timings by stage, top retrieval scores, model errors, and safe diagnostic logs.
- Only then consider reranking, query expansion, hybrid score tuning, deployment, or multi-user features.

The strongest next step is not necessarily adding more AI components. A broader corpus, labeled evaluation, and better failure analysis can improve a project more reliably than stacking new tools on top.

28. Viva questions and model answers

1. What is RAG?

Retrieval-Augmented Generation searches external knowledge at query time and uses retrieved evidence as context for the language model’s answer.

2. Why use both BM25 and embeddings?

BM25 catches exact technical terms and identifiers; embeddings catch semantic paraphrases. Their rankings are complementary.

3. Why RRF instead of adding raw scores?

BM25 scores and vector distances have different scales. RRF uses rank positions, so it can fuse lists without pretending their raw values are directly comparable.

4. Why chunk documents?

Chunking makes ranking more focused and keeps each embedding within model limits, while balancing precision against context completeness.

5. Why preserve heading metadata?

Headings supply context for isolated code or paragraphs and let the UI trace an answer back to a meaningful section.

6. What does a 384-dimensional embedding mean?

Each text is represented by a vector containing 384 numeric values. These dimensions are learned features; they do not correspond to 384 human-readable labels.

7. Does a high similarity score prove correctness?

No. It indicates vector proximity under the configured model/metric, not truth or entailment.

8. Why might the system refuse an answer?

No results or retrieval signals below configured thresholds trigger the insufficient-context path, avoiding an unnecessary guess.

9. What is the current evaluation limitation?

Six labeled questions and source-level labels form a smoke benchmark, not a representative evaluation of exact chunk relevance or answer correctness.

10. What would you improve first?

Expand and label an evaluation set, track retrieval and answer metrics separately, inspect failures, and then tune chunking, candidate counts, and thresholds.

29. Command reference and source links

Build and run

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

python .\src\ingest_docs.py --refresh
python .\src\process_docs.py
python .\src\index_docs.py

ollama pull llama3.2:3b
ollama serve

uvicorn src.api:app --host 127.0.0.1 --port 8000
streamlit run .\src\streamlit_app.py
```

Test and evaluate

```
python -m pytest -q
python -m src.evaluate_retrieval --k 5
```

Repository source of truth

Repository: https://github.com/ApexHyper2007/DocQuery-AI

README: https://github.com/ApexHyper2007/DocQuery-AI/blob/main/README.md

Implementation report: https://github.com/ApexHyper2007/DocQuery-AI/blob/main/PROJECT_REPORT.md

retrieval.py: https://github.com/ApexHyper2007/DocQuery-AI/blob/main/src/retrieval.py

answering.py: https://github.com/ApexHyper2007/DocQuery-AI/blob/main/src/answering.py

ollama_client.py: https://github.com/ApexHyper2007/DocQuery-AI/blob/main/src/ollama_client.py

api.py: https://github.com/ApexHyper2007/DocQuery-AI/blob/main/src/api.py

evaluate_retrieval.py: https://github.com/ApexHyper2007/DocQuery-AI/blob/main/src/evaluate_retrieval.py

This guide was prepared against the public `main` branch available on 20 October 2026. If the code changes, re-check repository files before relying on individual defaults or implementation descriptions.

> **A good project explanation connects each component to a problem it solves, explains its trade-offs, and acknowledges how its quality was evaluated.**